

Prompt Iteration Log

Task

FL-01 target task: ****Preparing a clear work/study update****

Original FL-01 definition: Use AI to turn raw notes about completed work, current tasks, blockers, and next steps into a concise professional update.

Done well: The update accurately covers completed work, current work, blockers/questions when relevant, and next steps; contains no invented information; and can be communicated in about 2 minutes or less.

Source Notes Used For Every Version

Completed:

- Finished W5 polite scraper.
- Scraper collects first 3 Books to Scrape catalogue pages.
- Found 60 unique book URLs.
- Wrote books.json, errors.json, and run-report.json.
- Rerun used cache and still produced 60 records.
- One fake URL was skipped and reported.
- Pushed code to GitHub.

Current:

- Packaging final submission.
- Checking README and final PDF.

Blockers:

- No major blocker.
- Need to make sure I submit the correct GitHub link and PDF.

Next:

- Submit GitHub link.
- Submit final PDF.
- Continue capstone work.

Version 0: Naive One-Line Prompt

Prompt

Write a work update from these notes: finished W5 scraper, got 60 books, made JSON files and report, pushed to GitHub, now submitting.

Output

I completed the W5 scraper assignment. It collects book data and creates JSON files and a report. I pushed the project to GitHub and am now preparing the final submission.

Note

This was short, but too thin. It missed the evidence that made the work trustworthy: first three catalogue pages, 60 unique URLs, cache rerun, broken-page handling, and exact next steps.

Version 1: Role Assignment

Named technique: ****role assignment****

Prompt

You are a clear technical mentor helping me write a short internship work update. Turn these notes into a professional update I can send to a supervisor.

Notes:

[source notes]

Output

Hi, I completed the W5 polite scraper assignment. The scraper collects data from the first three Books to Scrape catalogue pages, discovers 60 unique book URLs, and writes the results to JSON output files. I also added a run report and error file so the run is easier to verify.

I pushed the latest code to GitHub and am now packaging the final submission. There are no major blockers, but I am double-checking that I submit the correct GitHub link and PDF. Next, I will submit the work and continue with the capstone.

Note

The role made the tone more professional and supervisor-ready. It also improved completeness, but the output still did not mention cache behavior or the deliberate broken URL clearly enough.

Version 2: Context And Motivation

Named technique: ****context and motivation****

Prompt

You are helping me write a 2-minute internship update. The purpose is to show progress honestly, not to exaggerate. My supervisor should quickly understand what I completed, how I proved it worked, whether I am blocked, and what I will do next.

Use these notes only. Do not invent results.

Notes:

[source notes]

Output

This week I completed the W5 polite scraper work. The scraper is limited to the first three Books to Scrape catalogue pages, where it found 60 unique book URLs. It writes the validated records to `books.json`, records failures in `errors.json`, and saves run statistics in `run-report.json`.

I also checked that the scraper behaves safely on rerun: it used the cache and still produced 60 records instead of duplicating data. I added one fake URL to test failure handling, and the scraper skipped it and reported it without stopping the run.

I have pushed the code to GitHub and I am now packaging the final submission. I do not have a major blocker. My next steps are to submit the GitHub link and final PDF, then continue capstone work.

Note

Adding the motivation changed the output from a general update into a proof-oriented update. It now explains how the work was verified and avoids inflated language.

Version 3: Few-Shot Examples

Named technique: **few-shot examples**

Prompt

Write a concise internship update using the style shown in the examples.

Example 1:

Completed: I finished the auth route and tested missing-token and valid-token cases.

Proof: Swagger shows protected routes and login returns an access token.

Blocker: I still need to add the screenshot.

Next: I will save the screenshot and push it.

Example 2:

Completed: I moved the API to Postgres in Docker.

Proof: ``SELECT * FROM tasks;`` shows the same rows as ``GET /tasks``.

Blocker: No blocker.

Next: I will submit the GitHub link and PDF.

Now write the update from these notes:

[source notes]

Output

Completed: I finished the W5 polite scraper and pushed it to GitHub. It collects the first three Books to Scrape catalogue pages, discovers 60 unique book URLs, and writes ``books.json``, ``errors.json``, and ``run-report.json``.

Proof: The rerun used cache and still produced exactly 60 records. I also added one fake URL, and the scraper skipped it and recorded the failure instead of crashing.

Blocker: No major blocker. I only need to make sure I submit the correct GitHub link and final PDF.

Next: I will submit the GitHub link and PDF, then continue capstone work.

Note

The examples strongly improved structure. The output became easier to scan and matched the real update pattern, but it sounded slightly more template-like than my own voice.

Version 4: Output Structure

Named technique: **output structure**

Prompt

Create a work update from the notes below.

Output format:

- 4 short sections only: Completed, Proof, Blocker, Next.
- 1 to 2 sentences per section.
- Keep it plain and specific.

- Use only facts from the notes.
- Mention exact artifacts when useful.

Notes:

[source notes]

Output

Completed: I finished the W5 polite scraper and pushed the code to GitHub. It collects the first three Books to Scrape catalogue pages, finds 60 unique book URLs, and writes `books.json`, `errors.json`, and `run-report.json`.

Proof: I reran the scraper and confirmed it used cache while still producing 60 records. I also tested one fake URL, and the run skipped it and reported the failure without crashing.

Blocker: There is no major blocker. I am only double-checking that I submit the correct GitHub link and PDF.

Next: I will submit the GitHub link and final PDF. After that, I will continue capstone work.

Note

The explicit format kept the answer tight and prevented extra filler. This version was the most directly usable as a message, but it did not show the thinking process behind what should be included.

Version 5: Step Decomposition

Named technique: **step decomposition**

Prompt

Help me turn raw internship notes into a clear work update.

Steps:

1. Identify the completed work.
2. Identify the proof that the work actually ran.
3. Identify blockers or questions.
4. Identify next steps.
5. Write the final update in 120 words or less.

Rules:

- Use only the notes.
- Do not exaggerate.
- Keep the tone clear, honest, practical, and warm.

Notes:

[source notes]

Output

Completed: I finished the W5 polite scraper and pushed it to GitHub. It collects the first three Books to Scrape catalogue pages, finds 60 unique book URLs, and writes `books.json`, `errors.json`, and `run-report.json`.

Proof: I reran it and confirmed the cache worked while the output stayed at 60 records. I also tested one fake URL, which was skipped and reported without stopping the run.

Blocker: No major blocker. I am just checking that I submit the correct GitHub link and PDF.

Next: I will submit the link and PDF, then continue capstone work.

Note

Breaking the task into steps made the model select the right facts before drafting. The final answer stayed short but included the strongest evidence. This became the best version.

Version 6: Final Combined Prompt

Named techniques: ****role assignment, context and motivation, few-shot examples, output structure, step decomposition****

Prompt

You are a practical technical mentor helping an intern write a concise work update.

Purpose:

The update should help a supervisor understand progress quickly. It must be honest, specific, and based only on the notes. Do not invent results or make the work sound bigger than it is.

Voice:

Clear, honest, practical, warm, no buzzwords.

Process:

1. Identify completed work.
2. Identify proof that the work ran.
3. Identify blockers or questions.
4. Identify next steps.
5. Write the final update in 120 words or less.

Output format:

Completed:

Proof:

Blocker:

Next:

Example style:

Completed: I moved the API to Postgres in Docker.

Proof: ``SELECT * FROM tasks;`` shows the same rows as ``GET /tasks``.

Blocker: No blocker.

Next: I will submit the GitHub link and PDF.

Notes:

[source notes]

ChatGPT Output

Completed: I finished the W5 polite scraper and pushed it to GitHub. It collects the first three Books to Scrape catalogue pages, finds 60 unique book URLs, and writes ``books.json``, ``errors.json``, and ``run-report.json``.

Proof: I reran it and confirmed the cache worked while the output stayed at 60 records. I also tested one fake URL, which was skipped and reported without stopping the run.

Blocker: No major blocker. I am just checking that I submit the correct GitHub link and PDF.

Next: I will submit the link and PDF, then continue capstone work.

Note

Combining the techniques produced the strongest result. The role set the audience, the context explained why accuracy mattered, the example shaped the style, the structure kept it scannable, and the decomposition made the evidence appear in the right place.

Cross-Model Comparison

ChatGPT

ChatGPT followed the structure exactly and kept the update compact. It was strongest at preserving the requested headings and staying within the short professional-update style. Its main risk was smoothing the language so much that the update could become generic if the prompt did not force exact artifacts like `books.json`, `errors.json`, and `run-report.json`.

Claude

Claude was not available as a callable tool in this coding session, so I could not truthfully run the final prompt directly from here. To complete the live comparison manually, paste Version 6 into Claude with the same source notes. Based on the Anthropic prompting guidance, the prompt is Claude-ready because it is clear, gives context, assigns a role, includes an example, and specifies the output format.

Expected point to check in Claude's output: Claude may sound more natural and explanatory, but it may also use longer sentences unless the 120-word limit is enforced. The specific comparison to record after running it is whether Claude keeps all four headings, includes the fake-URL failure proof, and avoids adding details not in the notes.

Reusable Prompt Template

You are a practical technical mentor helping an intern write a concise work update.

Purpose:

The update should help [AUDIENCE] understand progress quickly. It must be honest, specific, and based only on the notes. Do not invent results or make the work sound bigger than it is.

Voice:

[VOICE CARD]

Process:

1. Identify completed work.
2. Identify proof that the work ran or was checked.
3. Identify blockers or questions.
4. Identify next steps.
5. Write the final update in [WORD LIMIT] words or less.

Output format:

Completed:

Proof:

Blocker:

Next:

Example style:

Completed: [one sentence about completed work]

Proof: [one sentence with concrete evidence, artifact, test, or result]

Blocker: [one sentence, or "No blocker"]
Next: [one sentence with the next action]

Notes:
[PASTE RAW NOTES HERE]

Sources Consulted

- Anthropic Prompt Engineering Interactive Tutorial: beginner chapters on basic prompt structure, clarity/directness, and assigning roles.
- Anthropic Claude prompting best practices: clear instructions, context, examples, XML/structured separation, and roles.
- OpenAI Help Center prompting best practices: clear and specific instructions, iterative refinement, tone guidance, task breakdown, and prioritizing focused requests.